



Facultad de Ciencias, UNAM

Ingeniería de Software.

Especificación de Requerimientos del Sistema

Iteración 4 - Consolidación Final del Sistema

Versión del Documento: 4.0

Versión del Sistema: Backend 4.0.0 | Frontend 3.0.0

Fecha de actualización: 25 de mayo de 2026

Equipo Lambda:

- Franco Anaya Juan Ramón - [318290733]
 - Jimenez Ruiz Brandon - [321246516]
- Arizmendi López Alexis de Jesús - [318176110]
- Méndez Ávila Luis Giovanni - [317143980]
- Díaz Quijada Alan Joseph - [424044259]



1. Introducción.

1.1 Propósito del Documento:

El presente documento es la evolución final del ERS entregado en la Iteración 3, actualizado para reflejar el estado real del sistema al cierre de la Iteración 4. Incorpora los tres nuevos casos de uso implementados: Reportar Mascota Desaparecida, Consultar Mascotas Desaparecidas y Editar Animal Publicado. Además se actualizan los requerimientos funcionales, el modelo de datos, las tecnologías y las limitaciones conocidas. Sirve como referencia para el equipo de desarrollo, nuevos integrantes y stakeholders.

1.2 Alcance del Sistema:

Adopta Perrito es una plataforma web que facilita la adopción de perros y gatos en México. A partir de la Iteración 4, el sistema también permite reportar y consultar mascotas desaparecidas, ampliando su utilidad más allá de la adopción. Los usuarios pueden registrarse, publicar animales en adopción, buscar y filtrar mascotas, ver su detalle, editar publicaciones propias y contactar a los publicadores. Los administradores pueden además eliminar animales y marcar adopciones completadas. El sistema no gestiona pagos ni trámites legales de adopción.

Alcance de la Iteración 4:

Esta iteración consolida el sistema con nuevos casos de uso como: Editar Animal Publicado, Reportar Mascota Desaparecida y Consultar Mascotas Desaparecidas. Con esto el sistema alcanza más casos de uso funcionales, cumpliendo con el mínimo requerido de 8 (sin contar los del módulo de usuario).

1.3 Definiciones relevantes:

- **Usuario Registrado:** Persona que ha creado una cuenta en la plataforma con nombre, correo electrónico y código postal
- **Publicador:** Usuario registrado que da en adopción uno o más animales.
- **Adoptante:** Usuario registrado interesado en adoptar un animal publicado en la plataforma.
- **Administrador:** Usuario con rol ADMIN. Tiene permisos para eliminar animales, marcar adopciones y gestionar el sistema.
- **Perfil del Animal:** Ficha con la información del animal en adopción: nombre, tipo, raza, descripción, fotografía y código postal.
- **Estado del Animal:** Valor que indica si un animal está disponible o adoptado.
- **Mascota Desaparecida:** Registro de un animal perdido, con datos de ubicación, contacto y estado de búsqueda.
- **Endpoint:** URL específica de la API REST que expone una funcionalidad del sistema.
- **Token:** Identificador de sesión generado al iniciar sesión, utilizado para autenticar



Equipo Lambda

- peticiones posteriores.
- **DTO:** Data Transfer Object. Objeto utilizado para transportar datos entre capas sin exponer el modelo de dominio.
- **ERS:** Especificación de Requerimientos del Sistema.
- **RF / RNF:** Requerimiento Funcional / Requerimiento No Funcional.
- **sessionStorage:** Almacenamiento del navegador utilizado para guardar el token de sesión del usuario de forma temporal.

2. Actores del sistema.

¿Cuáles son los actores en nuestro sistema?

Los actores representan los roles que interactúan con el sistema, ya sean personas o entidades externas.

1) Usuarios registrados:

Solo aquellos usuarios que estén registrados tendrán acceso a la aplicación y sus funcionalidades, es un requisito obligatorio el estar registrado.

1.1) Usuarios que publican animales:

Aquellos usuarios registrados de esta manera tendrán acceso a publicar fichas de adopción para que otros usuarios puedan interesarse en la mascota publicada. Un ejemplo claro podrían ser centros de adopción, clínicas veterinarias, refugios, etc. Este tipo de organizaciones son las que esperaríamos que utilicen este tipo de registro.

1.2) Usuarios interesados en adopcion:

Aquellos usuarios registrados de esta manera, son aquellos que están interesados en sumar una nueva mascota a su vida. Tendrán acceso a el feed donde se muestran aquellos animales disponibles, así como manifestar su interés en los mismos.

1.3) Administradores:

Este tipo de usuarios tienen privilegios extras sobre los demás, son usuarios con rol ADMIN, con sus privilegios extendidos: pueden eliminar animales del sistema y marcar mascotas como adoptadas. El rol se asigna directamente en la base de datos.

2) Sistema de correo electrónico:

Es el encargado de la comunicación automática. Cuando un usuario interesado en adoptar hace clic en el botón 'me interesa', este sistema entra en acción y notifica al publicador con la información de contacto del adoptante.

3) Sistema de registro y validación:

Es el encargado de validar los correos electrónicos, así también de validar que las contraseñas coinciden con las encriptadas.



3. Requerimientos funcionales.

Los requerimientos funcionales definen qué debe hacer un sistema, es decir, su comportamiento o acciones. Estos son obligatorios y se deben cumplir de alguna manera.

RF1) Registro de usuarios:

El sistema deberá permitir el registro de usuarios mediante nombre, correo electrónico válido y código postal.

RF2) Publicación de animales:

Aquellos usuarios que fueron registrados para publicar animales podrán crear fichas de adopción para sus animales que quieren dar en adopción. incluyendo nombre, descripción, fotografía, tipo, raza y código postal.

RF3) Autenticación de usuarios:

Solo aquellos usuarios que hayan sido autenticados tendrán acceso a la aplicación de manera completa.

RF4) Mostrar Interés:

Los usuarios interesados en alguna mascota, puedan mostrar su interés haciendo click en un botón que les permita expresar su interés y permita la comunicación entre los usuarios.

RF5) Visualización geográfica de mascotas:

Se debe mostrar a los usuarios interesados en adoptar un mapa interactivo con la ubicación aproximada de todas las mascotas disponibles. Además al darle click podrá tener más información de dicho animal.

RF6) Notificación de contacto:

Si un usuario manifiesta interés en una mascota , al propietario de la misma le llegará Información de contacto de la persona interesada.

RF7) Consulta detallada:

El sistema debe poder tener una opción “despliegue de vista detallada”, que permita a los usuarios obtener más información de la mascota que están consultando.

RF8) Filtro de búsqueda:

El sistema incluye un filtro de búsqueda para que los usuarios puedan buscar directamente mascotas con las especificaciones que más se adapten a lo que buscan.

RF9) Actualización de información del usuario:



Equipo Lambda

El sistema permite que un usuario autenticado actualice su información personal como el nombre, correo electrónico, código postal y/o contraseña.

RF10) Gestión de roles de usuario:

El sistema soporta dos roles: USER (por defecto) y ADMIN. El rol se almacena en la base de datos y se consulta al iniciar sesión para controlar el acceso a funcionalidades restringidas.

RF11) Eliminación de Animal (Admin):

Un usuario con rol Admin podrá eliminar el registro de un animal del sistema.

RF12) Marcar mascota como adoptada (Admin):

Un usuario con rol Admin podrá cambiar el estado de un animal de disponible a adoptado.

RF13) Edición de animal publicado:

El sistema permite al publicador de un animal (o a un ADMIN) editar la información de una publicación existente: nombre, especie, raza, descripción, código postal y fotografía.

RF14) Reporte de mascota desaparecida:

El sistema permite a los usuarios registrados reportar una mascota desaparecida, incluyendo nombre, especie, raza, edad, color, descripción, zona de desaparición, fecha de desaparición, teléfono de contacto e imagen opcional.

RF15) Consulta de mascotas desaparecidas:

El sistema permite a los usuarios consultar el listado de mascotas desaparecidas activas (aún no encontradas), con posibilidad de filtrar por zona y ver el detalle individual. Los admins (o el sistema) pueden marcar una mascota como encontrada.

4. Casos de uso.

CU1): Registro de usuario

Actores: Usuario (aún sin registrar)

Precondiciones:

- No debe estar registrado previamente
- Tener interés en poner en adopción o adoptar a un perro o gato
- Tener una cuenta de correo electrónico válida

Flujo Principal:

1. El usuario da click en el botón registrarse
2. Se abre un formulario para la captura de datos (nombre completo, código postal, correo electrónico, contraseña)
3. El usuario llena el formulario con los datos que se piden



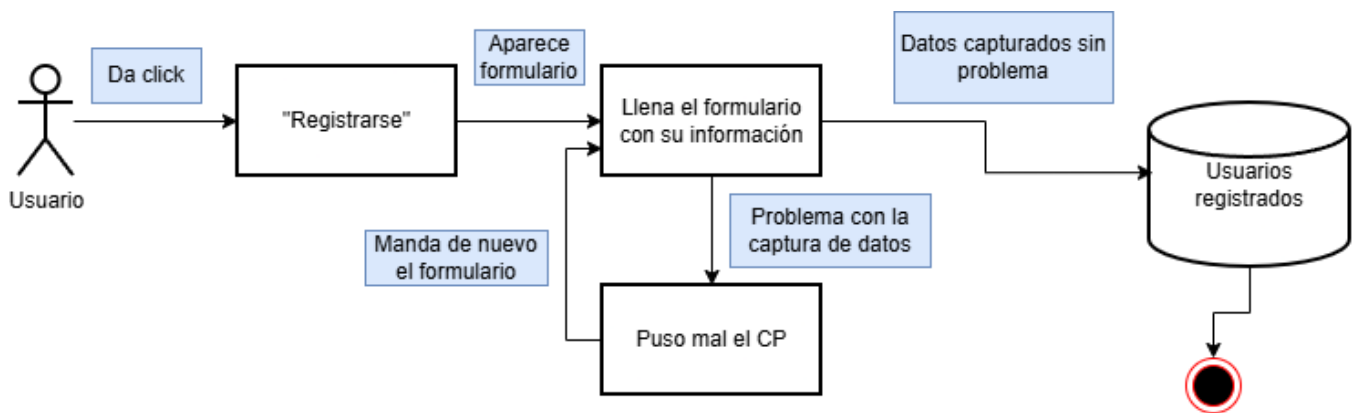
Equipo Lambda

4. El usuario da click en la opción terminar para concluir con su registro

Flujos Alternativos:

- A. El usuario da una dirección de correo ya registrada. El sistema notifica al usuario que esa cuenta ya ha sido utilizada .
- B. El usuario da un código postal inexistente. El sistema notifica al usuario que el código postal proporcionado no existe.

Diagrama:



CU2) Publicar Animal

Actores: Publicador.

Precondiciones:

- El usuario debe estar registrado
- El animal debe ser un perro o un gato

Flujo Principal:

1. El publicador da click en la opción publicar
2. El publicador ingresa los datos del animal (nombre, raza, especie, código postal, foto(s))
3. La publicación queda subida y visible para los interesados.
4. Se guarda el registro en el mapa para mostrar la ubicación del animal en adopción

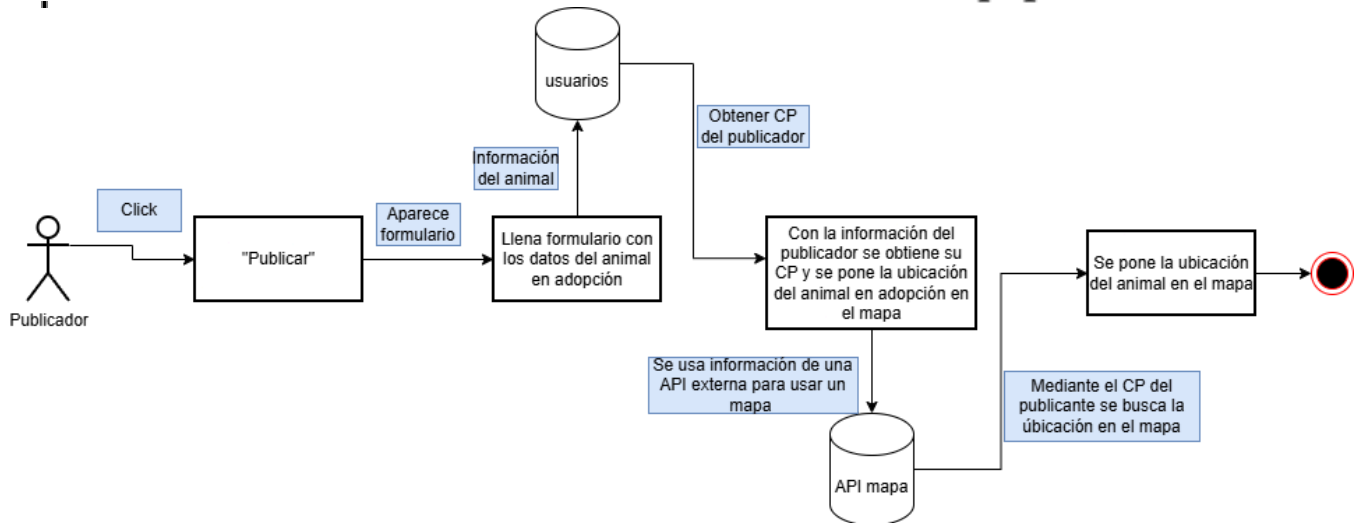
Flujo Alternativo:

- A. El publicador no sube completa la información del animal. En este caso no se procede con la publicación.

Diagrama:



Equipo Lambda



CU3) Mostrar interés en adoptar.

Actores: Publicador, Adoptante, Sistema, Animal

Precondiciones:

- Tanto el publicador como el adoptante deben estar registrados en la plataforma.
- El contacto debe ser mediante una publicación de adopción.

Flujo Principal:

1. El adoptante navega para encontrar algún animal en adopción
2. El adoptante da click en la ficha de algún animal para consultar sus datos
3. El adoptante da click en el botón “Estoy Interesado” cuando encuentra a un animal de su interés
4. La página inmediatamente notifica al publicador que hay un interesado en el animal que está dando en adopción
5. El sistema manda una alerta por correo electrónico al publicador con la información de contacto del adoptante.

Flujo Alternativo

- A. El adoptante únicamente mira la información del animal pero no da click en el botón “estoy interesado”. No sucede nada más.

CU4) Buscar mascota con filtros

- **Actores:** Usuario interesado en adopción.
- **Descripción:** El usuario filtra las mascotas disponibles para encontrar una que se ajuste a sus preferencias (especie, raza o cercanía).
- **Precondiciones:** El usuario debe haber iniciado sesión.
- **Flujo Principal:**
 1. El usuario accede a la sección del mapa o feed de mascotas.
 2. El usuario selecciona criterios de filtro (ej. Solo gatos, raza específica).



Equipo Lambda

3. El sistema procesa la solicitud y actualiza la vista.
4. El sistema muestra solo las mascotas que cumplen con los criterios.

CU5) Actualizar información del usuario

Actores: Usuario registrado

Precondiciones:

- El usuario debe haber iniciado sesión previamente.
- Tener un token válido almacenado en sessionStorage.

Flujo Principal:

1. El usuario accede a la vista de Actualización de Perfil.
2. El formulario muestra los campos modificables: nombre, correo electrónico, código postal y contraseña (todos opcionales).
3. El usuario modifica los campos deseados y envía el formulario.
4. El frontend realiza una petición PUT /usuarios al backend, enviando el token en el header Authorization y los nuevos datos en el cuerpo de la petición.
5. El backend valida el token, actualiza los datos del usuario en la base de datos y retorna el usuario actualizado (sin contraseña).
6. El frontend confirma al usuario que la actualización fue exitosa.

Flujos Alternativos:

- **A.** El token es inválido o ha expirado. El backend responde con error 401 y el frontend redirige al usuario al Login.
- **B.** El nuevo correo electrónico ya está en uso por otro usuario. El backend responde con error y el frontend muestra el mensaje correspondiente.

CU6) Ver detalle de una mascota

Actores: Usuario autenticado, Administrador

Precondiciones:

- El usuario debe haber iniciado sesión.
- La mascota debe existir en el sistema.

Flujo Principal:

1. El usuario da click en “Ver Detalle” desde la lista de resultados de búsqueda.
2. El frontend navega a /animales/:id y envía GET /animales/{id} con el token en el header Authorization.
3. El backend consulta la mascota por ID y retorna toda su información.
4. El frontend muestra nombre, especie, raza, código postal, descripción, foto y estado.



Equipo Lambda

5. Si el usuario es ADMIN, se muestran botones adicionales: “Marcar como Adoptado” y “Eliminar Mascota”

Flujos Alternativos:

- La mascota ya tiene estado ADOPTADO. El botón “Marcar como Adoptado” no se muestra.

Escenario de Error:

- La mascota con el ID proporcionado no existe. El backend responde 404 y el frontend muestra “No se encontró la mascota.”

CU7) Eliminar mascota

Actores: Usuario

Precondiciones:

- El usuario debe tener mascotas propias para eliminar
- La mascota debe existir en el sistema

Flujo Principal:

1. El usuario localiza la mascota propia (desde su perfil)
2. Da click en “Eliminar” y confirma la acción en el diálogo de confirmación
3. El frontend envía DELETE /animales/{id} con el token en el header Authorization
4. El frontend elimina la tarjeta de la vista y muestra confirmación, o redirige al catálogo si se eliminó desde el detalle

Flujos Alternativos:

- El usuario cancela la confirmación. No se realiza ninguna acción.

Escenarios de Error:

- El usuario intenta eliminar una mascota que no es suya. El backend responde 403 y el frontend muestra “No tienes permisos para eliminar mascotas.”
- La mascota no existe. El backend responde 404.

CU8) Marcar mascota como adoptada (Administrador)

Actores: Administrador

Precondiciones:

- El usuario debe tener rol ADMIN
- La mascota debe existir y tener estado DISPONIBLE



Equipo Lambda

Flujo Principal:

6. El administrador localiza la mascota
7. Da click en “Marcar como Adoptado” y confirma la acción
8. El frontend envía PUT /animales/{id}/adoptar con el token en el header Authorization
9. El backend valida el rol ADMIN, actualiza el campo estado a ADOPTADO y retorna el animal actualizado
10. El frontend refleja el nuevo estado en la vista (etiqueta verde “ADOPTADO” y botón oculto)

Flujos Alternativos:

2. El administrador cancela la confirmación. El estado no cambia.

Escenarios de Error:

- El usuario no tiene rol ADMIN. El backend responde 403.
- La mascota no existe. El backend responde 404.

CU9) Editar Animal Publicado

Actores: Publicador (dueño del animal), Administrador

Descripción: El publicador de un animal puede editar la información de su publicación existente. Un ADMIN también puede editar cualquier publicación.

Precondiciones:

- El usuario debe haber iniciado sesión.
- El animal debe existir en el sistema.
- El usuario debe ser el publicador del animal o tener rol ADMIN.

Flujo Principal (Normal):

1. El usuario localiza la mascota en la vista de detalle (/animales/:id).
2. Hace clic en el botón 'Editar' (visible solo para el publicador o un ADMIN).
3. El frontend navega a /animales/:id/editar y carga el formulario con los datos actuales del animal vía GET /animales/{id}.
4. El usuario modifica los campos deseados (nombre, especie, raza, descripción, código postal, foto) y hace clic en 'Guardar Cambios'.
5. El frontend envía PUT /animales/{id}/editar como multipart/form-data, incluyendo el token en el header Authorization.
6. El backend valida el token, verifica que el usuario sea el dueño o ADMIN, actualiza el registro y retorna el animal actualizado.
7. El frontend redirige al detalle del animal con los datos actualizados.

Flujo Alternativo:

- A. El usuario hace clic en 'Cancelar'. El frontend regresa al detalle del animal sin guardar



Equipo Lambda

- cambios.
- B. El usuario no selecciona una nueva imagen. La fotografía existente se conserva sin cambios.

Escenarios de Error:

- Error 401: el token es inválido o ha expirado. El frontend redirige al Login.
- Error 403: el usuario no es el dueño del animal ni ADMIN. El frontend muestra 'No tienes permisos para editar esta mascota.'
- Error 404: el animal no existe. El frontend muestra el mensaje correspondiente.
- Error 500: fallo al guardar la imagen. El backend retorna error y el frontend lo notifica al usuario.

Endpoints involucrados:

- GET /animales/{id} — carga datos actuales del animal.
- PUT /animales/{id}/editar — actualiza el animal (multipart/form-data).

CU10) Reportar Mascota Desaparecida

Actores: Usuario registrado

Descripción: El usuario puede reportar una mascota desaparecida proporcionando información detallada para facilitar su búsqueda.

Precondiciones:

- El usuario debe haber iniciado sesión.

Flujo Principal (Normal):

8. El usuario accede a la sección de mascotas desaparecidas.
9. Hace clic en 'Reportar Mascota Desaparecida'.
10. El frontend muestra un formulario con los campos: nombre, especie, raza (opcional), edad (opcional), color (opcional), descripción (opcional), zona de desaparición, fecha de desaparición, teléfono de contacto e imagen (opcional).
11. El usuario llena el formulario y hace clic en 'Enviar Reporte'.
12. El frontend envía POST /mascotas-desaparecidas con los datos en formato JSON y el token en el header Authorization.
13. El backend valida el token, registra la mascota desaparecida en la base de datos y retorna la entidad creada.
14. El frontend confirma al usuario que el reporte fue creado y redirige al listado de mascotas desaparecidas.

Flujo Alternativo:

- A. El usuario no llena los campos obligatorios (nombre, especie, zona de desaparición, fecha, teléfono). El frontend valida y muestra los campos faltantes.



Equipo Lambda

Escenarios de Error:

- Error 401: el token es inválido o ha expirado.
- Error 400: datos de entrada inválidos o campos obligatorios faltantes.
- Error 500: fallo interno del servidor al guardar el reporte.

Endpoints involucrados:

- POST /mascotas-desaparecidas — crea el reporte de mascota desaparecida.

CU11) Consultar Mascotas Desaparecidas

Actores: Usuario registrado, Administrador

Descripción: El usuario puede consultar el listado de mascotas desaparecidas activas, filtrar por zona, ver el detalle de una mascota en particular y (si tiene permisos) marcarla como encontrada.

Precondiciones:

- El usuario debe haber iniciado sesión.

Flujo Principal (Normal):

15. El usuario accede a la sección 'Mascotas Desaparecidas'.
16. El frontend envía GET /mascotas-desaparecidas y muestra el listado de mascotas con estado encontrada=false.
17. El usuario puede opcionalmente filtrar por zona enviando GET /mascotas-desaparecidas/buscar?zona=<valor>.
18. El usuario hace clic en una mascota para ver su detalle vía GET /mascotas-desaparecidas/{id}.
19. El detalle muestra: nombre, especie, raza, edad, color, descripción, zona, fecha, teléfono de contacto e imagen.
20. Si el usuario es ADMIN (o el sistema), se muestra el botón 'Marcar como Encontrada'.
21. Al hacer clic en dicho botón, el frontend envía PUT /mascotas-desaparecidas/{id}/encontrada y el sistema actualiza el campo encontrada a true.

Flujo Alternativo:

- A. No hay mascotas desaparecidas activas. El frontend muestra un mensaje informativo.
- B. La búsqueda por zona no arroja resultados. El frontend muestra un mensaje informativo.

Escenarios de Error:

- Error 401: el token es inválido.
- Error 404: la mascota desaparecida con el ID proporcionado no existe.

Endpoints involucrados:



Equipo Lambda

- GET /mascotas-desaparecidas — lista todas las mascotas activas (encontrada=false).
- GET /mascotas-desaparecidas/{id} — detalle de una mascota.
- GET /mascotas-desaparecidas/buscar?zona=<valor> — filtro por zona.
- PUT /mascotas-desaparecidas/{id}/encontrada — marca como encontrada.

5. Requerimientos No Funcionales.

RNF1-Usabilidad

La interfaz debe ser responsiva y permitir que el registro completo de un animal en adopción no exceda los 10 clics por parte del usuario.

RNF2-Usabilidad

Los mensajes de error del sistema deben ser claros y comprensibles, indicando al usuario el campo exacto que causó el fallo en la captura de datos.

RNF3-Rendimiento

El sistema deberá tener la capacidad de soportar al menos 200 usuarios simultáneos navegando sin presentar degradación en los tiempos de respuesta.

RNF4-Disponibilidad

El sistema debe garantizar un tiempo de actividad (uptime) del 99.9% anual, asegurando la disponibilidad de la plataforma para los usuarios interesados.

RNF5-Seguridad

Las credenciales de acceso de los usuarios (contraseñas) deberán almacenarse en la base de datos mediante un algoritmo de cifrado o hashing seguro.

RNF6-Seguridad del Token

El token de sesión debe almacenarse únicamente en sessionStorage del navegador y enviarse en el header Authorization en cada petición que requiera autenticación. El token debe invalidarse en la base de datos al ejecutar el cierre de sesión.

RNF7 – Control de acceso basado en roles

Las operaciones administrativas (eliminar animal, marcar como adoptado) deben estar restringidas exclusivamente a usuarios con rol ADMIN, tanto en el backend (validación en el controller) como en el frontend (ocultamiento condicional de controles).

6. Tecnologías Utilizadas.

A continuación se describen las tecnologías utilizadas en el desarrollo del sistema durante la



Equipo Lambda

Iteración 2, junto con la justificación de su elección. Las tecnologías de backend se mantienen desde la Iteración 1.

6.1 Backend (sin cambios respecto a Iteración 1)

Kotlin — Lenguaje de programación

Lenguaje moderno sobre la JVM con sintaxis concisa y seguridad de tipos. Elegido por su interoperabilidad con Java, soporte nativo en Spring Boot y capacidad de reducir código repetitivo en comparación con Java puro.

Spring Boot — Framework de backend

Framework que simplifica la configuración y el arranque de aplicaciones REST en la JVM. Elegido por ser el estándar de la industria, su amplia documentación y su integración nativa con JPA y Maven.

PostgreSQL — Base de datos relacional

Sistema gestor de base de datos relacional de código abierto. Elegido por su robustez, soporte completo del estándar SQL y amplia adopción en proyectos de producción.

JPA / Hibernate — Capa de persistencia

JPA es la especificación estándar para mapeo objeto-relacional en Kotlin/Java. Permite interactuar con la base de datos usando objetos Kotlin en lugar de escribir SQL directamente.

Maven — Gestión de dependencias

Herramienta que gestiona las dependencias del proyecto y automatiza la compilación. Elegido por ser el estándar en proyectos Spring Boot y su integración con IntelliJ IDEA.

Dotenv-Kotlin — Variables de entorno

Librería que permite leer credenciales sensibles desde un archivo .env en lugar de escribirlas directamente en el código, evitando exponer información en el repositorio.

Postman — Cliente HTTP para pruebas

Herramienta utilizada para probar los endpoints de la API REST y verificar el comportamiento del sistema de extremo a extremo.

IntelliJ IDEA — Entorno de desarrollo (IDE)

IDE principal del equipo para desarrollo en Kotlin y Spring Boot, con soporte integrado para Maven, Git y bases de datos.



Equipo Lambda

6.2 Frontend (sin cambios respecto a Iteración 2)

React 19 — Framework de frontend

Biblioteca de JavaScript para la construcción de interfaces de usuario mediante componentes reutilizables. Elegida por su amplia adopción en la industria, su ecosistema maduro y la familiaridad del equipo con el paradigma basado en componentes.

Vite — Herramienta de construcción y servidor de desarrollo

Bundler moderno que proporciona un servidor de desarrollo con recarga instantánea (Hot Module Replacement) y tiempos de compilación muy reducidos. Elegido por su velocidad superior respecto a Create React App y su configuración mínima para proyectos React.

React Router DOM 7 — Enrutamiento del lado del cliente

Biblioteca estándar para el manejo de rutas en aplicaciones React de una sola página (SPA). Permite navegar entre vistas (Login, Register, Home, Perfil, Actualización) sin recargar la página, protegiendo rutas que requieren autenticación.

Fetch API — Comunicación con el backend

API nativa del navegador utilizada para realizar peticiones HTTP al backend. Se eligió la Fetch API nativa en lugar de bibliotecas externas como Axios para mantener el número de dependencias al mínimo y aprovechar las capacidades ya incorporadas en los navegadores modernos.

sessionStorage — Almacenamiento del token de sesión

Mecanismo de almacenamiento del navegador utilizado para guardar el token de sesión del usuario. A diferencia de localStorage, sessionStorage limita la vida del token a la duración de la pestaña activa, reduciendo el riesgo de sesiones abandonadas de larga duración.

7. Arquitectura General.

La arquitectura del sistema no presenta cambios estructurales respecto a la Iteración 3. El sistema continúa operando con dos componentes independientes (frontend y backend) que se comunican mediante peticiones HTTP REST.

7.1 Componentes del sistema

Componente	Responsabilidad	Tecnologías
Frontend	Interfaz de usuario que el usuario final utiliza desde el	React 19, Vite, React



Equipo Lambda

	navegador. Gestiona la navegación entre vistas, la validación de formularios, el almacenamiento del token y la comunicación con el backend.	Router DOM 7, Fetch API
Backend	API REST que procesa las peticiones del frontend, aplica la lógica de negocio, valida tokens y persiste la información en la base de datos. Organizado en capas: Controller, Service, Repository.	Kotlin, Spring Boot, JPA/Hibernate, PostgreSQL, Maven
Base de Datos	Almacenamiento persistente de la información de los usuarios (con rol) y mascotas, incluyendo nombre, correo, código postal, contraseña hashada y token de sesión.	PostgreSQL

7.2 Flujo de una petición en el sistema

Cuando el usuario interactúa con alguna vista del frontend, el flujo general es:

1. El usuario realiza una acción en el navegador (llenar un formulario, hacer clic en un botón).
2. El frontend construye la petición HTTP correspondiente, incluyendo el token en el header Authorization cuando la operación lo requiere.
3. La petición llega al Controller del backend, que extrae y valida los datos de entrada.
4. El Controller delega la operación al Service correspondiente.
5. El Service aplica las reglas de negocio y llama al Repository.
6. El Repository ejecuta la operación en PostgreSQL mediante JPA.
7. La respuesta regresa al frontend en formato JSON.
8. El frontend actualiza la vista del usuario o muestra el mensaje de error correspondiente.

Esta separación garantiza que cada componente tenga una única responsabilidad, facilitando el mantenimiento y la incorporación de nuevos desarrolladores.

7.3 Flujo de Autenticación

Sin cambios respecto a la Iteración 3. El sistema utiliza tokens UUID generados en el login, almacenados en sessionStorage en el frontend y en la columna token de la tabla usuario en la base de datos. Los endpoints protegidos validan el token y adicionalmente verifican el rol del usuario cuando la operación lo requiere (ADMIN).

Inicio de sesión

El usuario ingresa sus credenciales (correo electrónico y contraseña) en la vista de Login. El frontend envía estas credenciales al backend, que valida la contraseña hashada y, si son correctas, genera un token de sesión único que queda almacenado en la base de datos y se retorna al frontend.



Equipo Lambda

Almacenamiento y uso del token

Una vez recibido el token, el frontend lo guarda en sessionStorage del navegador. A partir de ese momento, todas las peticiones que requieran autenticación (consultar perfil, actualizar información, cerrar sesión) incluyen este token en el header Authorization con el formato Bearer <token>.

Protección de vistas y endpoints

En el frontend, al cargar cualquier vista protegida (Home, Perfil, Actualización de Perfil), se verifica la existencia del token en sessionStorage. Si no existe, el usuario es redirigido automáticamente a la vista de Login. En el backend, los endpoints protegidos validan el token recibido contra la base de datos antes de procesar la petición.

Token inválido o inexistente

Si el backend recibe un token que no corresponde a ninguna sesión activa, responde con un error de autenticación (HTTP 401). El frontend, al recibir este error, elimina cualquier token almacenado en sessionStorage y redirige al usuario al Login. Cuando el usuario cierra sesión explícitamente, el token es invalidado en la base de datos y eliminado del sessionStorage.

8. Modelo Conceptual de Datos.

En las Iteraciones 1 y 2 el sistema maneja únicamente la entidad Usuario. En la Iteración 3 el sistema gestiona dos entidades: Usuario y Animal. A partir de la Iteración 4, el sistema gestiona tres entidades: Usuario, Animal y MascotaDesaparecida.

Entidad Usuario:

Atributo	Tipo	Descripción	Seguridad
id_usuario	Entero (serial)	Identificador único autogenerado por la BD.	—
nombre	Texto (100)	Nombre completo del usuario.	—
correo	Texto (100)	Correo electrónico único del usuario.	No se expone en endpoints públicos.
codigo_postal	Texto (10)	Código postal. Usado para filtrar animales en el mapa.	—
contraseña	Texto(100)	Credencial de acceso del usuario, almacenada como hash SHA-256.	Nunca se retorna en respuestas, debe



Equipo Lambda

			cifrarse.
token	Texto(100)	Identificador de sesión activa, generado en el login.	Se invalida al cerrar sesión. No se expone innecesariamente.
rol	Texto(20)	Rol del usuario: USER (default) o ADMIN	Controla acceso a operaciones restringidas

Consideraciones generales de seguridad:

- La contraseña nunca debe retornarse en ninguna respuesta de la API.
- Las contraseñas se almacenan hasheadas con SHA-256.
- El token de sesión debe invalidarse en la base de datos al ejecutar el cierre de sesión.
- El correo electrónico no debe exponerse en endpoints de búsqueda pública.

Entidad Animal:

Atributo	Tipo	Descripción	Seguridad
id_animal	Entero (serial)	Identificador único autogenerado por la BD.	—
nombre	Texto (100)	Nombre del animal.	—
especie	Texto (50)	Tipo de animal: Perro o Gato.	—
raza	Texto (100)	Raza del animal (opcional)	—
descripción	Texto	Descripción libre del animal (opcional)	—
foto_url	Texto(255)	URL de la fotografía del animal (opcional)	—
código_postal	Texto(10)	Ubicación aproximada del animal	—
estado	Texto(20)	Estado del animal: DISPONIBLE (default) o ADOPTADO	Solo admins pueden modificarlo
id_usuario	FK→Usuario	Referencia al publicador del animal	—



Entidad MascotaDesaparecida:

Atributo	Tipo	Descripción	Seguridad
id	Entero (serial)	Identificador único autogenerado por la BD.	—
nombre	Texto (100)	Nombre del animal.	—
especie	Texto (50)	Tipo de animal: Perro o Gato.	—
raza	Texto (100)	Raza del animal (opcional)	—
edad	Entero(opcional)	Edad en años del animal.	—
color	Entero(opcional)	Color predominante del animal.	—
descripción	Texto	Descripción libre del animal (opcional)	—
foto_url	Texto(255)	URL de la fotografía del animal (opcional)	—
zonaDesapari cion	Texto(100)	Zona donde desapareció.	—
fechaDesapar icion	LocalDate	Fecha en que desapareció.	—
telefonoCont acto	Texto(20)	Teléfono de contacto del reportante.	—
encontrada	Boolean	Indica si la mascota ya fue encontrada. Default: false.	—
usuario	FK→Usuario	Referencia al que realizo el reporte	—

9. Versionamiento Formal.

9.1 Versionamiento del código

Repositorio	Iteración	Versión (tag)
Backend	Iteración 1	1.0.0



Equipo Lambda

Backend	Iteración 2	2.0.0
Frontend	Iteración 2	1.0.0
Backend	Iteración 3	3.0.0
Frontend	Iteración 3	2.0.0
Backend	Iteración 4	4.0.0
Frontend	Iteración 4	3.0.0

9.2 Versionamiento del Documento

Versión	Fecha	Cambios principales
1.0	17 de marzo de 2026	Documento inicial. Backend con endpoints de registro, login, logout y consulta de perfil.
2.0	10 de abril de 2026	Incorporación del frontend (React + Vite). Nuevo endpoint PUT /usuarios. Secciones 4.3 Tecnologías, 4.4 Arquitectura, 4.5 Flujo de Autenticación.
3.0	8 de mayo de 2026	Expansión del sistema con módulo de animales. Nuevos casos de uso. Campo rol en Usuario, entidad Animal con estado. Nuevos Requerimientos. Modelo de datos actualizado. Limitaciones revisadas.
4.0	25 de mayo de 2026	Consolidación final. Nuevos casos de uso como Editar Animal, Reportar Mascota y Consultar Mascotas Desaparecidas. Nueva entidad MascotaDesaparecida. Versiones Backend 4.0.0 y Frontend 3.0.0.